

EMILIANO VOLPINO

Desarrollador Full Stack — Diseño de Sistemas & Backend | Node.js · TypeScript · PostgreSQL

Buenos Aires, Argentina | +54 9 11 4989-1169 | volpinoemiliano@gmail.com | linkedin.com/in/emilianovulpino | github.com/Wolpi066

PERFIL

Desarrollador Full Stack que diseña y construye sistemas completos, no features aisladas. Llevo plataformas de la pizarra a producción y las mantengo funcionando: reglas de negocio, modelo de datos, arquitectura de backend, integraciones, despliegue e infraestructura. Hoy opero dos sistemas en producción: un motor de automatización comercial sobre WhatsApp que reemplaza unas 3 horas diarias de trabajo manual, y una plataforma e-commerce B2B/B2C con punto de venta desarrollada para un cliente. Mi terreno fuerte es lo que pasa cuando el sistema se cruza con la realidad: concurrencia, consistencia transaccional, límites de recursos y fallas de producción diagnosticadas hasta la causa raíz. Ecosistema TypeScript/Node.js sobre PostgreSQL, con TDD. Inglés C1.

EXPERIENCIA

Desarrollador Full Stack — Diseño y Arquitectura de Software (Freelance) · Vixit Labs

2026 – Presente · Buenos Aires (remoto)

- Ownership técnico end-to-end de productos a medida: plataformas e-commerce B2B/B2C, sistemas de punto de venta y motores de automatización. Único responsable de cada proyecto, de la primera reunión al deploy, el monitoreo y el soporte en producción.
- Decisiones de arquitectura: modelo de datos relacional, separación de responsabilidades del backend, estrategia de concurrencia y consistencia, diseño de contratos de API y elección del stack de infraestructura según la carga real y el presupuesto del cliente.
- Traducción de reglas de negocio a sistemas: convierto procesos operativos ambiguos —precios diferenciados, mínimos de compra, conciliaciones, flujos de aprobación— en modelos de datos y lógica ejecutable y auditable.
- Diseñé “Código Cero”, programa de formación en desarrollo de software asistido por IA, orientado a la transición de programador a diseñador de sistemas (lanzamiento 2026). Actualmente dicto mentoría técnica 1:1.

Analista de Operaciones — Claims · Pro Insurance Solutions S.A.

Febrero 2026 – Presente

- Gestión y auditoría de 100 a 120 borderós mensuales de siniestros, garantizando la integridad, trazabilidad y confidencialidad de los datos en los sistemas internos.
- Detección y resolución de discrepancias sobre volúmenes altos de registros mediante análisis con Excel avanzado, bajo métricas de calidad y normativa interna estricta.
- Conocimiento del dominio de seguros desde adentro del proceso de claims: cuellos de botella operativos, fricción manual y los controles que un sistema de gestión de siniestros necesita resolver. Trabajo con Operations & People identificando tareas repetitivas candidatas a automatización.

Operaciones de E-commerce y Ventas Online · Independiente

2021 – 2025

- Operación integral de venta online punta a punta: publicación, stock, logística y atención al cliente. Es la razón por la que hoy modelo bien sistemas de e-commerce: conozco el negocio que el software tiene que representar, no solamente el código.

PROYECTOS DESTACADOS

Vacuum Engine — Motor de automatización comercial B2B/B2C · Vixit Labs

Node.js · TypeScript · Express · Prisma · PostgreSQL (Supabase) · Baileys 7 (WebSocket) · Puppeteer · Angular · GitHub Actions · VPS Oracle Cloud (Ubuntu, nginx, systemd) · Vercel

- Plataforma en producción que centraliza una operación comercial completa: intercepta ofertas de proveedores en canales y grupos de WhatsApp, sincroniza a diario los catálogos de cuatro proveedores (3.400+ productos con precio, stock e imagen) y permite redistribuir cualquier oferta desde una única bandeja web. Reemplaza unas 3 horas diarias de monitoreo, copiado y reenvío manual.

- Reescribí la integración de WhatsApp migrando de whatsapp-web.js + Chrome headless a Baileys 7 sobre WebSocket, eliminando el navegador del servidor: la RAM libre pasó de ~55 MB a ~380 MB en una VPS de 1 GB.
- Diagnosticué la causa raíz de un fallo de autenticación recurrente en producción: al terminar el proceso sin cerrar el socket, el estado criptográfico de la sesión (ratchet del protocolo Signal) quedaba a medio escribir y WhatsApp rechazaba la reconexión con un 401, forzando re-vinculación manual. Resuelto con un handler de SIGTERM/SIGINT que cierra el socket y drena las escrituras pendientes antes de salir.
- Rediseñé la arquitectura de ingesta moviendo el scraping del backend a GitHub Actions programadas: saqué Chrome del servidor y desacoplé la extracción de datos del servicio en línea. Incluyó diagnosticar por qué los runners no alcanzaban la base —la conexión directa de Supabase es IPv6-only— y redirigirla al pooler. Los scrapers en Puppeteer manejan scroll infinito, lazy-load y reintentos: ajustar la intercepción de recursos corrigió una extracción que se cortaba en 225 productos y la llevó al catálogo completo de ~1.638.
- Optimización de rendimiento con resultado medido: carga completa de la aplicación de 5-6 s a 1,1 s. Caché en memoria con patrón stale-while-revalidate y precalentado al arranque, refresco debounced que colapsa una ráfaga de 20 mensajes en una sola consulta, proyección selectiva de columnas, gzip en nginx (1,12 MB a 253 KB) y, en el frontend Angular, caché con shareReplay, trackBy y la eliminación de un getter que re-filtraba 3.346 productos en cada ciclo de detección de cambios.
- Endpoint de envío optimista que confirma en 0,48 s y despacha en segundo plano, con timeout en toda llamada externa y degradación automática a texto plano si falla el envío de la imagen.
- Operación y resiliencia: servicio systemd con reinicio automático y prioridad frente al OOM killer, zram con compresión 3,3:1 para sobrevivir el límite de memoria, HTTPS vía nginx y Let's Encrypt, purga horaria de datos con más de 48 h y un health-check cada 30 minutos que alerta por e-mail ante caída del backend o de la sesión de WhatsApp.

Plataforma E-commerce B2B/B2C + Punto de Venta — Distribuidora Harry's · proyecto para cliente

React (Vite) · Node.js · Express · PostgreSQL (SQL nativo) · Tailwind CSS · Mercado Pago API · Resend · node-cron · Vercel / Railway / Cloudflare

- Arquitectura de roles dinámicos: diseñé un modelo que sirve precios, mínimos de compra y reglas de descuento diferenciados para consumidor final y cliente mayorista sobre un único catálogo, evitando la duplicación de productos y los problemas de consistencia asociados.
- Punto de venta físico sincronizado en tiempo real con el inventario web, con soporte de pagos divididos: un mismo stock atacado concurrentemente desde el mostrador y desde la web sin desincronización.
- Control de concurrencia: diseñé el pipeline de ingesta masiva de catálogo por importación Excel resistente a condiciones de carrera, combinando caché reactiva en memoria, bloqueos transaccionales a nivel de fila (FOR UPDATE) e inserciones idempotentes (ON CONFLICT DO NOTHING). Dimensionado para el catálogo de ~1.700 SKU del cliente.
- Blindaje transaccional: ningún importe se confía al cliente. El backend revalida y recalcula de forma autónoma precios, stock y cupones antes de confirmar cada operación, cerrando la superficie de manipulación desde el frontend.
- Webhooks de pago con verificación criptográfica de firma, reportes financieros automatizados por e-mail con node-cron, y despliegue en producción con separación de entornos: frontend en Vercel, backend y base en Railway, DNS y protección en Cloudflare.

Breath Shop — E-commerce Full Stack · github.com/Wolpi066/Breath

Angular 17+ · PHP (MVC nativo, sin framework) · MySQL · JWT

- SPA reactiva en Angular 17+ aprovechando Signals y componentes standalone para gestión de estado de alto rendimiento sin librerías externas.
- Ingeniería de un backend MVC propio en PHP nativo: ruteo, controladores y capa de acceso a datos escritos desde cero, con autenticación stateless vía JWT y hashéo de contraseñas con Bcrypt.
- Stock en tiempo real que sincroniza el carrito contra el inventario para evitar sobreventa en compras concurrentes, sobre un esquema relacional con consultas parametrizadas contra inyección SQL.

El Cisma — Motor de juego Survival Horror · github.com/Wolpi066/ElCisma

C++ · Raylib · Tau (testing framework)

- Game loop propio y gestión manual de memoria en C++ sin garbage collection, con control explícito del ciclo de vida de cada entidad y del presupuesto de frame.
- Máquinas de estados finitos para la IA de enemigos, con transiciones por proximidad y estímulos sonoros, y suite de tests unitarios bajo TDD sobre colisiones y lógica de inventario.

COMPETENCIAS TÉCNICAS

Backend y datos: Node.js, TypeScript, Express.js, PostgreSQL (SQL nativo y Prisma), Supabase, MySQL, diseño de esquemas relacionales, control de concurrencia y transacciones, APIs REST, JWT, Webhooks

Frontend: React.js (Vite), Angular 17+ (Signals, standalone components, RxJS), JavaScript ES6+, Tailwind CSS, Bootstrap

Arquitectura y prácticas: Diseño de sistemas transaccionales, arquitectura MVC, modelado de reglas de negocio, estrategias de caché y optimización de rendimiento, TDD, integración de APIs y pasarelas de pago, automatización de procesos, Scrum

Infraestructura y DevOps: Linux (VPS Oracle Cloud, Ubuntu), nginx, systemd, Let's Encrypt, GitHub Actions (CI y jobs programados), Vercel, Railway, Cloudflare, Git/GitHub, Postman, monitoreo y alertas, web scraping con Puppeteer

Otros lenguajes: C++, PHP, Java, Python, SQL · **IA aplicada:** LLMs y prompt engineering como herramienta de diseño, revisión y aceleración del ciclo de desarrollo

EDUCACIÓN

Tecnicatura Superior en Programación — UCES · En curso, egreso previsto diciembre 2026

Analista de Sistemas — Instituto Tecnológico ORT · 2020 – 2022 · **Cursos:** Ciberseguridad y Ethical Hacking (CoderHouse, 2025), Desarrollo Web y JavaScript (CoderHouse, 2022)

Idiomas: Español (nativo) · Inglés (C1 – Avanzado, escrito y conversacional)